

Mật mã học - NT219.Q11.ANTN.1 - Lab 3

Báo cáo thực hành Lab 3

Bài tập 1:

Đề bài

Viết chương trình (bằng ngôn ngữ C/C++) để thực hiện các yêu cầu sau:

1. Số nguyên tố:

- Sinh ngẫu nhiên các số nguyên tố có độ dài 2 byte, 8 byte và 32 byte.
- Kiểm tra số nguyên bất kỳ nhỏ hơn $2^{89} - 1$ có phải là số nguyên tố hay không.

2. Ước số chung lớn nhất:

Tính ước số chung lớn nhất của hai số nguyên “lớn” tùy ý (càng lớn càng tốt trong phạm vi chương trình có thể xử lý).

3. Lũy thừa mô-đun:

Tính giá trị $a^x \bmod p$.

Chương trình phải có khả năng xử lý các trường hợp có số mũ lớn ($x > 40$), ví dụ: $7^{40} \bmod 19$.

Không sử dụng bất kỳ thư viện cryptography nào.

Giải

Ở bài này mình sẽ khai báo thư viện `bigint` để làm việc trên những con số lớn.

Phần 1: Số nguyên tố

Ở mục này, ta cần làm hai bước - 1 là tạo số ngẫu nhiên, 2 là kiểm tra tính nguyên tố.

Mình tạo số ngẫu nhiên bằng cách sử dụng random bằng bit (bit-level random), sau đó check prime bằng thuật toán Rabin-Miller, cụ thể là thuật toán đơn định (https://wiki.vnoi.info/algo/algebra/primality_check.md)

https://wiki.vnoi.info/algorithm/algebra/primality_check.md))

Hàm random bit để tạo số lẻ:

```
bigint randomOddBigint(int bytes) {
    vector<unsigned char> data(bytes);
    for (int i = 0; i < bytes; i++) data[i] = rand() & 0xFF;

    if (bytes > 1)
        data[0] |= 0x80;
    data.back() |= 1;

    bigint result = 0;
    for (unsigned char c : data)
        result = result * to_bigint(256) + to_bigint(c);

    return result;
}
```

Hàm check random:

```

bigint pow(bigint base, bigint exp, bigint mod) {
    bigint res = bigint(1);
    base %= mod;

    while (exp > bigint(0)) {
        if (exp % bigint(2) == bigint(1))
            res = (res * base) % mod;
        exp /= bigint(2);
        base = (base * base) % mod;
    }
    return res;
}

bool test(bigint a, bigint n, bigint k, bigint m){
    bigint mod = pow(a, m, n);
    bigint one = 1;
    if (mod == one || mod == n - one)
        return true;

    for (int l = 1; l < k; ++l){
        mod = (mod * mod) % n;
        if (mod == n - one)
            return true;
    }
    return false;
}

bool MillerRabin(bigint n){
    static vector<bigint> checkSet = {2, 3, 7, 11, 13};
    bigint two = 2;
    if (n < two)
        return false;

    bigint k = 0, m = n - to_bigint(1);
    while (m % to_bigint(2) == to_bigint(0)){
        m /= 2;
        k++;
    }

    for (bigint a : checkSet){
        if (a >= n) continue;
        if (!test(a, n, k, m))
            return false;
    }

    return true;
}

```

Khi xem qua phần code trên bạn sẽ thấy mình để `checkSet` rất ít số bởi vì ở hàm `genPrime`, mình sẽ check tính chia hết với những số nguyên tố nhỏ để đặt bộ lọc giúp bỏ bớt những số không thỏa và tăng tính chính xác của số nguyên tố hơn:

```
const vector<bigint> smallPrimes = {
    3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53,
    59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127,
    131, 137, 139, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199,
    211, 223, 227, 229, 233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283,
    293, 307, 311, 313, 317, 331, 337, 347, 349, 353, 359, 367, 373, 379, 383,
    389, 397, 401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461, 463, 467,
    479, 487, 491, 499, 503, 509, 521, 523, 541
};

bigint genPrime(int bytelength){
    int count = 0;
    while(true){
        bigint p = randomOddBigint(bytelength);
        bool isComposite = false;
        for (bigint prime : smallPrimes) {
            if (p == prime) {
                isComposite = false;
                break;
            }

            if (p % prime == to_bigint(0)) {
                isComposite = true;
                break;
            }
        }
        if (isComposite)
            continue;
        if (MillerRabin(p))
            return p;
        count++;
        if(count == 10)
            cout << "Checked 10 random numbers!!!\n";
    }
}
```

Thật vậy, mình đã tạo vài số nguyên tố và kiểm tra thì nó cho ra kết quả với độ chính xác cao, tuy nhiên về hiệu suất thì với số bit càng lớn, tốc độ sinh số nguyên tố càng lâu.

Sau đây là 1 vài ví dụ sinh số nguyên tố 2 byte, 8 byte, 32 byte, kiểm tra số nguyên tố và cách mình double check những số này là bằng đoạn code python sử dụng thư viện pycryptodome :3

```
# genPrime.py
from Crypto.Util.number import getPrime
for i in range(3):
    print(getPrime(90-i))

# 778910092719973123873611623
# 575482606982477339332775363
# 275939704016341849195231097

# checkPrime.py
from Crypto.Util.number import isPrime
for _ in range(3):
    p = int(input("Enter your prime: "))
    if isPrime(p):
        print(f"{p} is a prime")
    else:
        print(f"{p} is not a prime")
```

Ví dụ:

```
This is my program related to Number Theory

Here is the menu selection:
1. PRIME NUMBERS
2. GREATEST COMMON DIVISOR
3. MODULAR EXPONENTIATION
4. Exit the program
Enter your choice (1, 2, 3, 4): 1

-----PRIME NUMBERS-----
--Generate random primes with bit-length--
Enter your byte selection: 2
Here is your 2-byte prime number: 54547
Check prime numbers: 535503437018065389967269619
535503437018065389967269619 is a prime number

Here is the menu selection:
1. PRIME NUMBERS
2. GREATEST COMMON DIVISOR
3. MODULAR EXPONENTIATION
4. Exit the program
Enter your choice (1, 2, 3, 4): 1

-----PRIME NUMBERS-----
--Generate random primes with bit-length--
Enter your byte selection: 8
Here is your 8-byte prime number: 12622376025184634287
Check prime numbers: 451441641597854652509094523
451441641597854652509094523 is a prime number

Here is the menu selection:
1. PRIME NUMBERS
2. GREATEST COMMON DIVISOR
3. MODULAR EXPONENTIATION
4. Exit the program
Enter your choice (1, 2, 3, 4): 1

-----PRIME NUMBERS-----
--Generate random primes with bit-length--
Enter your byte selection: 32
Here is your 32-byte prime number: 78144243335092953773358867187368064842142225241944491137658637127340209700943
Check prime numbers: 327818691897586922698485401
327818691897586922698485401 is a prime number
```

Check:

```
Enter your prime: 54547
54547 is a prime
Enter your prime: 12622376025184634287
12622376025184634287 is a prime
Enter your prime: 78144243335092953773358867187368064842142225241944491137658637127340209700943
78144243335092953773358867187368064842142225241944491137658637127340209700943 is a prime
```

Full code mình để ở đây: [Github](https://github.com/r1muru2006/Number-Theory) (https://github.com/r1muru2006/Number-Theory).

Phần 2: Ước số chung lớn nhất

Ở phần tiếp theo này, mình sử dụng thuật toán Euclid với độ phức tạp là $O(\log(\min(x, y)))$ khi thực hiện tính toán $\gcd(x, y)$ và đây là script:

```

#include <bits/stdc++.h>
#include "bigint.h"

using namespace std;

bigint myGCD(bigint a, bigint b) {
    if (a == to_bigint(0))
        return b;
    return myGCD(b % a, a);
}

int main() {
    bigint x, y;
    cout << "\n---GREATEST COMMON DIVISOR---\n";
    cout << "Enter first number: "; cin >> x;
    cout << "Enter second number: "; cin >> y;
    bigint res2 = myGCD(x, y);
    cout << "Result: gcd(" << x << ", " << y << ") = " << res2 << endl;
    return 0;
}

```

Ví dụ:

```

Here is the menu selection:
1. PRIME NUMBERS
2. GREATEST COMMON DIVISOR
3. MODULAR EXPONENTIATION
4. Exit the program
Enter your choice (1, 2, 3, 4): 2

---GREATEST COMMON DIVISOR---
Enter first number: 16481162970505983551
Enter second number: 15841604293862234929
Result: gcd(16481162970505983551, 15841604293862234929) = 4255827341

```

Phần 3: Lũy thừa mô-đun

Để tính giá trị $a^x \bmod p$ thì mình viết hàm `pow` sử dụng thuật toán nhân lũy thừa bằng bình phương để tính nhanh lũy thừa, sau đây là code:

```
#include <bits/stdc++.h>
#include "bigint.h"

using namespace std;

bigint pow(bigint base, bigint exp, bigint mod) {
    bigint res = bigint(1);
    base %= mod;

    while (exp > bigint(0)) {
        if (exp % bigint(2) == bigint(1))
            res = (res * base) % mod;
        exp /= bigint(2);
        base = (base * base) % mod;
    }
    return res;
}

int main() {
    bigint b, e, p;
    cout << "\n---MODULAR EXPONENTIATION---\n";
    cout << "Enter base: "; cin >> b;
    cout << "Enter exponent: "; cin >> e;
    cout << "Enter modulo: "; cin >> p;
    bigint res3 = pow(b, e, p);
    cout << "The value of " << b << "^" << e << " mod " << p << " is: " << res3 <<
    return 0;
}
```



Mình thấy đề bài yêu cầu còn nhẹ đô =))))))

```
Here is the menu selection:
1. PRIME NUMBERS
2. GREATEST COMMON DIVISOR
3. MODULAR EXPONENTIATION
4. Exit the program
Enter your choice (1, 2, 3, 4): 3

----MODULAR EXPONENTIATION----
Enter base: 7
Enter exponent: 40
Enter modulo: 19
The value of 7^40 mod 19 is: 7

Here is the menu selection:
1. PRIME NUMBERS
2. GREATEST COMMON DIVISOR
3. MODULAR EXPONENTIATION
4. Exit the program
Enter your choice (1, 2, 3, 4): 3

----MODULAR EXPONENTIATION----
Enter base: 1423464362
Enter exponent: 467516623643
Enter modulo: 54762964386263646573575623
The value of 1423464362^467516623643 mod 54762964386263646573575623 is: 24623756528057147840107468
```

Bài tập 2

Đề bài

Làm quen với thuật toán RSA bằng cách sử dụng các công cụ tự chọn hỗ trợ để thực hiện các thí nghiệm sau:

1. Sử dụng các công cụ tự chọn, xác định khóa công khai (PU) và khóa bí mật (PR) nếu:
 - $p_1 = 11$, $q_1 = 17$, $e_1 = 7$ (hệ thập phân)
 - $p_2 = 20079993872842322116151219$, $q_2 = 676717145751736242170789$, $e_2 = 17$ (hệ thập phân)
 - $p_3 = F7E75FDC469067FFDC4E847C51F452DF$, $q_3 = E85CED54AF57E53E092113E62F436F4F$, $e_3 = 0D88C3$ (hệ thập lục phân)
2. Sử dụng khóa được tạo từ p_1 , q_1 , e_1 để mã hóa và giải mã bản rõ $M = 5$ trong hai trường hợp với code C/C++:
 - Mã hóa nhằm bảo mật thông tin (Confidentiality).
 - Mã hóa nhằm xác thực (Authentication).

Giải

Giới thiệu sơ qua về RSA:

Tạo khóa

- Chọn hai số nguyên tố lớn p và q .
- Tính $n = p \times q$ và $\phi(n) = (p - 1)(q - 1)$.
- Chọn số e sao cho $1 < e < \phi(n)$ và $\gcd(e, \phi(n)) = 1$.
- Tính d là nghịch đảo của e theo modulo $\phi(n)$: $d \times e \equiv 1 \pmod{\phi(n)}$.
→ Khóa công khai: (e, n) , khóa bí mật: (d, n) .

Mã hóa

$C \equiv M^e \pmod{n}$, với M là bản rõ và C là bản mã.

Giải mã:

$M \equiv C^d \pmod{n}$ để thu lại bản gốc.

Ở phần 1, mình chỉ phải giải quyết bài toán Modular multiplicative inverse để tìm ra d khi biết p, q, e và từ đó tạo ra cặp khóa RSA: $e \cdot d \equiv 1 \pmod{\phi(n)}$

Ý tưởng là sử dụng thuật toán Euclid mở rộng, lấy hai số nguyên a và b , sau đó tìm ƯCLN của chúng và tìm x, y thỏa mãn: $ax + by = \gcd(a, b)$

Với bài toán: $A \cdot B \equiv 1 \pmod{M}$, ta đặt $b = M$ trong công thức trên.

Vì ta biết A và M nguyên tố cùng nhau nên: $Ax + My = 1$

Lấy modulo M ở cả hai vế, ta được: $Ax \equiv 1 \pmod{M}$

Như vậy, x mà ta tìm được là nghịch đảo modulo của A .

```

bigint gcdExtended(bigint a, bigint b, bigint* x, bigint* y){
    if (a == to_bigint(0)) {
        *x = 0, *y = 1;
        return b;
    }

    bigint x1, y1;
    bigint gcd = gcdExtended(b % a, a, &x1, &y1);

    *x = y1 - (b / a) * x1;
    *y = x1;

    return gcd;
}

bigint inv(bigint A, bigint M){
    bigint x, y;
    bigint g = gcdExtended(A, M, &x, &y);
    if (g != to_bigint(1)){
        cout << "Inverse doesn't exist";
        return 0;
    }
    bigint res = (x % M + M) % M;
    return res;
}

```

Cuối phần 1, ta có 1 bộ dữ liệu hệ thập lục phân nên ta sẽ viết hàm để chuyển đổi nó:

```

int hexCharToInt(char c) {
    if (c >= '0' && c <= '9')
        return c - '0';

    if (c >= 'A' && c <= 'F')
        return c - 'A' + 10;

    return 0;
}

bigint hexToBigint(string hex) {
    bigint res = bigint(0);
    bigint pow16 = bigint(1);

    for (int i = hex.length() - 1; i >= 0; i--) {
        int digit = hexCharToInt(hex[i]);
        res += (bigint(digit) * pow16);
        pow16 *= 16;
    }

    return res;
}

```

Hàm tạo khóa RSA:

```

vector<pair<bigint, bigint>> create_RSA_key(bigint p, bigint q, bigint e){
    bigint n = p*q;
    bigint phi = (p - to_bigint(1))*(q - to_bigint(1));
    assert(big_gcd(e, phi) == to_bigint(1) && "e and phi are not coprime");
    assert(e < phi && "e must be less than phi");
    bigint d = inv(e, phi);

    vector<pair<bigint, bigint>> key;
    key.push_back({e, n}); // public key
    key.push_back({d, n}); // private key
    return key;
}

```

Ở phần 2, đề yêu cầu mã hóa theo 2 phương thức: bảo mật thông tin và xác thực.

Sự khác biệt cơ bản của 2 phương thức này là việc sử dụng bộ cặp khóa khác nhau để thực hiện các hoạt động của chúng:

```

int main() {
    bigint p1 = 11, q1 = 17, e1 = 7;
    vector<pair<bigint, bigint>> RSA_key1 = create_RSA_key(p1, q1, e1);

    bigint M = 5, n = RSA_key1[0].second, e = RSA_key1[0].first, d =
RSA_key1[1].first;
    // CASE 1: CONFIDENTIALITY
    // - A use Public Key (e, n) to encrypt
    // - B use Private Key (d, n) to decrypt
    cout << "\n\nSTARTING ENCRYPTION PROCESS:\n";
    bigint C = big_pow(M, e) % n;
    cout << "A sends B the encrypted message: " << C << '\n';
    bigint M_dec = big_pow(C, d) % n;
    cout << "B decrypts the message and gets: " << M_dec << '\n';
    assert(M == M_dec);
    cout << "DECRYPTED SUCCESSFULLY!!!\n\n";

    // CASE 2: AUTHENTICATION
    // - A use Private Key (d, n) to signing
    // - B use Public Key (e, n) to verifying
    cout << "STARTING AUTHENTICATION PROCESS:\n";
    bigint S = big_pow(M, d) % n;
    cout << "A signs the message and sends B: " << S << '\n';
    bigint M_ver = big_pow(S, e) % n;
    cout << "B verifies the message and gets: " << M_ver << '\n';
    assert(M == M_ver);
    cout << "VERIFIED SUCCESSFULLY!!!\n\n";

    return 0;
}

```

Bài tập 3

Đề bài

Viết chương trình minh hoạ cách RSA hoạt động đơn giản, đáp ứng các yêu cầu:

1. Sinh cặp khóa (PU, PR) từ các đầu vào “hợp lệ” p, q, e được cung cấp; nếu không có p, q, e thì sinh cặp khóa ngẫu nhiên. Lưu ý kích thước cặp khóa càng lớn càng tốt.
2. Dùng các khóa đã sinh để mã hoá/giải mã thông điệp. Thông điệp là chuỗi (bao gồm icon): I love pinanek 🥰 i hate pinanek 😡
Không được sử dụng bất kỳ thư viện mật mã nào.

Giải

Ở phần 1, mình sẽ làm 2 case, 1 case là nhập vào p, q, e và tạo key RSA thử nếu không thỏa thì chuyển qua case 2 và tạo random cặp khóa RSA dựa vào độ dài bytes của số nguyên tố p, q mà người dùng cung cấp:

```
cout << "---Generate RSA key pair---\n";
cout << "Enter the prime p: "; cin >> p;
cout << "Enter the prime q: "; cin >> q;
cout << "Enter the public exponent e: "; cin >> e;

if(MillerRabin(p) && MillerRabin(q) && p*q > to_bigint(4294967296LL))
    RSA_key = create_RSA_key(p, q, e);

if (RSA_key.empty()) {
    cout << "---Invalid input---\n";
    cout << "\n---Generate random RSA key pair---\n";
    int length_p, length_q;
    e = 65537;
    cout << "Enter your bytelength of prime p: "; cin >> length_p;
    cout << "Enter your bytelength of prime q: "; cin >> length_q;
    do {
        bigint p_rd = genPrime(length_p);
        bigint q_rd = genPrime(length_q);
        if (p_rd == q_rd) continue;

        cout << "Successfully generated random key\n";
        cout << "Random p = " << p_rd << endl;
        cout << "Random q = " << q_rd << endl;
        cout << "Set e = " << e << endl;
        RSA_key = create_RSA_key(p_rd, q_rd, e);
    } while (RSA_key.empty());
}
cout << "Public Key: (" << RSA_key[0].first << ", " << RSA_key[0].second << ")\n";
cout << "Private Key: (" << RSA_key[1].first << ", " << RSA_key[1].second << ")\n";
bigint n = RSA_key[0].second, d = RSA_key[1].first;
```



Ở phần 2, mình thấy là icon này nằm ở kiểu dữ liệu kí tự có độ rộng 32 bit, vì vậy mình sẽ chuyển theo đúng kiểu dữ liệu là `char32_t` rồi sang `bigint` để mã hóa và giải mã bằng RSA

```

wstring_convert<codecvt_utf8<char32_t>, char32_t> convert;
u32string msg = convert.from_bytes("I love pinanek 🐼 i hate pinanek 🐼");
cout << "\nOriginal message: I love pinanek 🐼 i hate pinanek 🐼\n";

vector<bigint> enc;
for (char32_t c : msg){
    bigint M = to_bigint(static_cast<ll>(c));
    bigint C = pow(M, e, n);
    enc.push_back(C);
}
cout << "The encrypted message: ";
for (const bigint& num : enc) {
    cout << num << " ";
}
cout << endl;

u32string recovered;
for (bigint num : enc) {
    bigint M_rec = pow(num, d, n);
    ll rec = stoll(M_rec.to_string());
    recovered += static_cast<char32_t>(rec);d
}
string msg_rec = convert.to_bytes(recovered);
cout << "The decrypted message: " << msg_rec << '\n';
assert(msg_rec == "I love pinanek 🐼 i hate pinanek 🐼");
cout << "---Successful encryption/decryption process---\n";

```

Bài tập 4

Đề bài

Viết một ứng dụng mã hóa (C++, sử dụng thư viện Crypto++) để đáp ứng các yêu cầu sau:

1. Hỗ trợ thuật toán: ECDSA (Elliptic Curve Digital Signature Algorithm).
2. Chức năng: có thể ký (signing) và xác minh (verify) chữ ký số.
3. ECC curve: phải chọn từ các đường cong chuẩn (standard curves).
4. Thông điệp để ký / Chữ ký để xác minh:
 - Dữ liệu đầu vào được đọc từ file (sử dụng tên file).
 - Hỗ trợ tiếng Việt (sử dụng setmode, mã hóa UTF-16).
5. Khóa bí mật / khóa công khai:
 - Các khóa được tải từ file (áp dụng cho cả hai chức năng).

- Khóa công khai phải có độ dài tối thiểu 256 bit.

Sau đó, hãy tạo một bộ dữ liệu đầu vào với các kích thước khác nhau (ít nhất 3 kích thước).

Chạy chương trình của bạn 100 lần, lấy thời gian trung bình để thực hiện.

Tổng hợp kết quả trong bảng gồm các cột:

- Kích thước dữ liệu đầu vào
- Chế độ hoạt động (ký / xác minh)
- Thời gian mã hóa (encryption/signing time)
- Thời gian giải mã (decryption/verification time).

Giải

Đây là script của tôi:


```

#include <assert.h>
#include <fstream>
#include <iostream>
#include <string>
#include <chrono>

#include "cryptopp/pem.h"
#include "cryptopp/pem_common.h"
#include "cryptopp/osrng.h"
#include "cryptopp/hex.h"
#include "cryptopp/aes.h"
#include "cryptopp/integer.h"
#include "cryptopp/sha.h"
#include "cryptopp/filters.h"
#include "cryptopp/files.h"
#include "cryptopp/eccrypto.h"
#include "cryptopp/oids.h"

using namespace CryptoPP;
using namespace std;

bool GenPrivKey(const OID& oid, ECDSA<ECP, SHA1>::PrivateKey& key);
bool GenPubKey(const ECDSA<ECP, SHA1>::PrivateKey& pkey, ECDSA<ECP, SHA1>::PublicK

void SavePrivKey(const string& fname, const ECDSA<ECP, SHA1>::PrivateKey& key);
void SavePubKey(const string& fname, const ECDSA<ECP, SHA1>::PublicKey& key);
void LoadPrivKey(const string& fname, ECDSA<ECP, SHA1>::PrivateKey& key);
void LoadPubKey(const string& fname, ECDSA<ECP, SHA1>::PublicKey& key);

bool SignMsg(const ECDSA<ECP, SHA1>::PrivateKey& key, const string& msg, string& s
bool VerifyMsg(const ECDSA<ECP, SHA1>::PublicKey& key, const string& msg, const st

int isPemPub(string fname);
int isPemPriv(string fname);

int isPemPub(string fname) {
    ifstream f(fname);
    if (!f.is_open()) return 2;
    string line;
    getline(f, line);
    if (line.find("-----BEGIN PUBLIC KEY-----") == string::npos)
        return 0;
    while (getline(f, line)) {
        if (line.find("-----END PUBLIC KEY-----") != string::npos)
            return 1;
    }
}

```

```

    return 0;
}

int isPemPriv(string fname) {
    ifstream f(fname);
    if (!f.is_open()) return 2;
    string line;
    getline(f, line);
    if (line.find("-----BEGIN EC PRIVATE KEY-----") == string::npos)
        return 0;
    while (getline(f, line)) {
        if (line.find("-----END EC PRIVATE KEY-----") != string::npos)
            return 1;
    }
    return 0;
}

int main(int argc, char* argv[]) {
#ifdef __linux__
    std::locale::global(std::locale("C.UTF-8"));
#endif
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
#endif

    if (argc == 1) {
        cerr << "Usage: sol4 <mode>\n";
        cerr << "Mode: keygen, sign, verify\n";
        return 1;
    }

    int choice = 0;
    string mode = argv[1];
    if (mode == "keygen") choice = 1;
    else if (mode == "sign") choice = 2;
    else if (mode == "verify") choice = 3;
    else {
        cerr << "Invalid mode: " << mode << endl;
        return 1;
    }

    switch (choice) {
        case 1: {
            if (argc != 5) {
                cerr << "Usage: sol4 keygen <Private_key_filename> <Public_key_fil";
                cerr << "Supported file type: ber, pem.\n";
                return 1;
            }
        }
    }
}

```

```

    }

    string privFile = argv[2];
    string pubFile = argv[3];
    string fType = argv[4];

    auto start = chrono::high_resolution_clock::now();

    ECDSA<ECP, SHA1>::PrivateKey pkey;
    ECDSA<ECP, SHA1>::PublicKey pubkey;

    bool res = GenPrivKey(ASN1::secp160r1(), pkey);
    assert(res);

    res = GenPubKey(pkey, pubkey);
    assert(res);

    if (fType == "ber") {
        SavePrivKey(privFile, pkey);
        SavePubKey(pubFile, pubkey);
    } else if (fType == "pem") {
        FileSink pub(pubFile.data(), true);
        PEM_Save(pub, pubkey);

        FileSink priv(privFile.data(), true);
        PEM_Save(priv, pkey);
    } else {
        cerr << "Invalid file type!!!\n";
        return 1;
    }

    auto end = chrono::high_resolution_clock::now();
    auto dur = chrono::duration_cast<chrono::milliseconds>(end - start).count();

    cout << "Successfully saved ECDSA keys.\n";
    cout << "Time for key generation: " << dur << " milliseconds\n";
    break;
}

case 2: {
    if (argc != 5) {
        cerr << "Usage: sol4 sign <Private_key_file> <file/screen> <message>\n";
        cerr << "Example: sol4 sign priv.pem file mess.txt\n";
        return 1;
    }

    string keyFile = argv[2];
    string inType = argv[3];
    string inSrc = argv[4];

```

```

int inputMode = (inType == "screen") ? 1 : (inType == "file") ? 2 : 0;
if (!inputMode) {
    cerr << "Invalid option!!\n";
    return 1;
}

ECDSA<ECP, SHA1>::PrivateKey key;
int fType = isPemPriv(keyFile);

if (fType == 0)
    LoadPrivKey(keyFile, key);
else if (fType == 1) {
    FileSource fs(keyFile.data(), true);
    PEM_Load(fs, key);
} else {
    cerr << "Fail to load ECDSA key!!!\n";
    return 1;
}

cout << "Successfully loaded ECDSA key.\n";

string msg, sig;
if (inputMode == 1)
    msg = inSrc;
else
    FileSource(inSrc.data(), true, new StringSink(msg));

auto start = chrono::high_resolution_clock::now();
SignMsg(key, msg, sig);
auto end = chrono::high_resolution_clock::now();

auto dur = chrono::duration_cast<chrono::microseconds>(end - start).count();
cout << "Successfully signed message.\n";
cout << "Time for signing: " << dur << " microsecond\n";

string sigHex;
StringSource(sig, true, new HexEncoder(new StringSink(sigHex)));
cout << "Signature (hex): " << sigHex << endl;
break;
}

case 3: {
    if (argc != 5) {
        cerr << "Usage: sol4 verify <Public_key_file> <file/screen> <message>\n";
        cerr << "Example: sol4 verify pub.pem file mess.txt\n";
        return 1;
    }
}

```

```

string keyFile = argv[2];
string inType = argv[3];
string inSrc = argv[4];

int inputMode = (inType == "screen") ? 1 : (inType == "file") ? 2 : 0;
if (!inputMode) {
    cerr << "Invalid option!!\n";
    return 1;
}

ECDSA<ECP, SHA1>::PublicKey key;
int fType = isPemPub(keyFile);

if (fType == 0)
    LoadPubKey(keyFile, key);
else if (fType == 1) {
    FileSource fs(keyFile.data(), true);
    PEM_Load(fs, key);
} else {
    cerr << "Fail to load ECDSA key!!!\n";
    return 1;
}

cout << "Successfully loaded ECDSA key.\n";

string msg, sig, sigHex;
cout << "Enter signature to verify: ";
cin >> sigHex;

StringSource(sigHex, true, new HexDecoder(new StringSink(sig)));

if (inputMode == 1)
    msg = inSrc;
else
    FileSource(inSrc.data(), true, new StringSink(msg));

auto start = chrono::high_resolution_clock::now();
bool res = VerifyMsg(key, msg, sig);
auto end = chrono::high_resolution_clock::now();

auto dur = chrono::duration_cast<chrono::microseconds>(end - start).count();
cout << "Time for verifying: " << dur << " microsecond\n";

cout << (res ? "Verify successfully!!!" : "Verify fail!!!") << endl;
break;
}

default:
    cerr << "Invalid option!!!\n";

```

```

        return 1;
    }

    return 0;
}

bool GenPrivKey(const OID& oid, ECDSA<ECP, SHA1>::PrivateKey& key) {
    AutoSeededRandomPool prng;
    key.Initialize(prng, oid);
    assert(key.Validate(prng, 3));
    return key.Validate(prng, 3);
}

bool GenPubKey(const ECDSA<ECP, SHA1>::PrivateKey& pkey, ECDSA<ECP, SHA1>::PublicKey
    AutoSeededRandomPool prng;
    assert(pkey.Validate(prng, 3));
    pkey.MakePublicKey(pubkey);
    assert(pubkey.Validate(prng, 3));
    return pubkey.Validate(prng, 3);
}

void SavePrivKey(const string& fname, const ECDSA<ECP, SHA1>::PrivateKey& key) {
    key.Save(FileSink(fname.c_str(), true).Ref());
}

void SavePubKey(const string& fname, const ECDSA<ECP, SHA1>::PublicKey& key) {
    key.Save(FileSink(fname.c_str(), true).Ref());
}

void LoadPrivKey(const string& fname, ECDSA<ECP, SHA1>::PrivateKey& key) {
    key.Load(FileSource(fname.c_str(), true).Ref());
}

void LoadPubKey(const string& fname, ECDSA<ECP, SHA1>::PublicKey& key) {
    key.Load(FileSource(fname.c_str(), true).Ref());
}

bool SignMsg(const ECDSA<ECP, SHA1>::PrivateKey& key, const string& msg, string& s
    AutoSeededRandomPool prng;
    sig.clear();

    StringSource(msg, true,
        new SignerFilter(prng,
            ECDSA<ECP, SHA1>::Signer(key),
            new StringSink(sig)
        )
    );
    return !sig.empty();
}

```

```
}
```

```
bool VerifyMsg(const ECDSA<ECP, SHA1>::PublicKey& key, const string& msg, const st  
    bool res = false;  
    StringSource(sig + msg, true,  
        new SignatureVerificationFilter(  
            ECDSA<ECP, SHA1>::Verifier(key),  
            new ArraySink((CryptoPP::byte*)&res, sizeof(res))  
        )  
    );  
    return res;  
}
```